TECHNICAL SPECIFICATION MANUAL

DATABRICKS DATA, AI/ML REFERENCE ARCHITECTURE

Modernized Data Infrastructure and Low-Latency Fraud Detection Simulation

Executive Brief: This specification outlines the target-state Data & AI architecture designed to support high-volume transaction processing and sub-second anomaly detection at **BrickCompany**, a leading financial payments processor. Simulating a scale corresponding to roughly 8% of national transactions, this reference architecture establishes how migrating from fragmented AWS EMR and on-premise Oracle Exadata legacy estates to a unified **Databricks Lakehouse on AWS** resolves governance siloing, unlocks centralized end-to-end data lineage, and implements an ultra-fast, continuous machine learning model scoring loop.

1. The Architectural Shift: Legacy Migration Overview

Historically, high-scale transaction processors have struggled with highly fragmented analytics landscapes. In the legacy environment, operational overhead was elevated due to concurrent maintenance of AWS EMR Hadoop clusters for distributed pipeline runs and on-premises Oracle Exadata/Data Warehouses for structured business intelligence. Decentralized data governance limited complete column-level lineage tracking and hampered the rapid deployment of real-time machine learning features.

By adopting a unified **Databricks Lakehouse Platform**, BrickCompany assessed over **6,500 legacy ETL pipelines** and successfully eliminated over **3,000 redundant batch jobs**, consolidating them into an optimized medallion data flow governed end-to-end by Unity Catalog. The ingestion pipeline costs were reduced by approximately **40%**, and overall query performance across analytical and business intelligence workloads experienced a **3x improvement**.

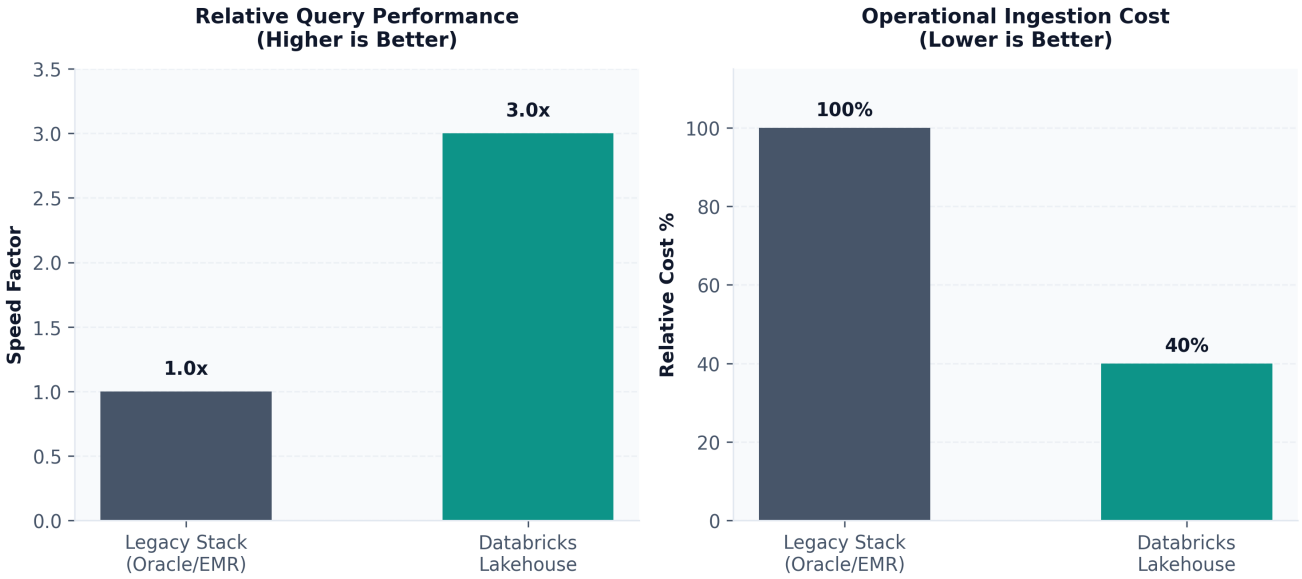


Figure 1. Operational Improvements and Cost Efficiencies post Databricks Migration

2. Legacy Refactoring: Oracle PL/SQL to Spark SQL

A major bottleneck in payment processing architectures is the existence of legacy database procedural packages. For BrickCompany, hundreds of business-critical calculations were locked within complex Oracle PL/SQL packages. These procedural blocks were refactored into highly optimized distributed PySpark pipelines and Databricks SQL scripts. This refactoring was accelerated using GenAI-driven code translation pipelines, resulting in a **60% operational cost reduction** due to license retirement and compute isolation.

Key Architectural Parameter	Legacy Architecture	Modernized Lakehouse Platform
Pipeline Footprint	6,500+ heterogeneous, disjointed batch pipelines across EMR & Oracle	3,500 consolidated, unified pipelines on Delta Lake; 3,000+ redundant jobs retired
Compute Engine	AWS EMR (Hadoop MapReduce / Spark batch) & On-premise Oracle Exadata	Unified Serverless Databricks Compute, auto-scaling Photon execution engine
Query Performance SLA	Highly variable query times, database locking on complex reports	Average 3x query performance improvement, consistent concurrent analytical SLA
Ingestion Pipeline Cost	Baseline (100% cloud & license allocation)	40% cost reduction (~60% total operational cost savings)
Data Catalog & Lineage	Decentralized, manual Excel tracking, no automated column lineage	Fully automated metadata catalogs, central lineage, single pane of glass

Table 1. Structural Comparison: Legacy Financial Architecture vs. Databricks Lakehouse

3. High-Throughput Real-Time Fraud Engine

To protect financial transaction margins, fraud detection must operate strictly in-line. Credit card transactions must be evaluated and authorized or declined within a tight sub-second window (SLA limit of 300ms). Historically, this sub-second requirement forced organizations to establish separate, highly complex streaming tools (e.g., separate Flink or Storm clusters) outside their primary data platform. This bifurcation doubled operational complexity and created severe sync lag.

To address this, BrickCompany leverages **Apache Spark Structured Streaming Real-Time Mode (RTM)** on Databricks. RTM enables continuous, event-by-event stream processing with end-to-end latencies down to tens of milliseconds (typically achieving **250ms end-to-end**). This allows the entire pipeline—from ingestion, stateful velocity feature engineering, dictionary-based enrichment, machine learning model scoring, and routing—to run on a single unified platform using familiar Spark APIs.

4. Real-Time Feature Engineering and ML Inference

The core of the payment fraud detection pipeline relies on the continuous calculation of stateful features, such as tracking cumulative purchase frequency or transactional volume for a given card within sliding temporal windows. This is achieved at scale using the state-of-the-art **transformWithState (TWS)** API in Spark. TWS allows the engine to manage per-key state directly, applying Time-To-Live (TTL) based expiration configurations to automatically purge inactive cardholder state and control memory growth.

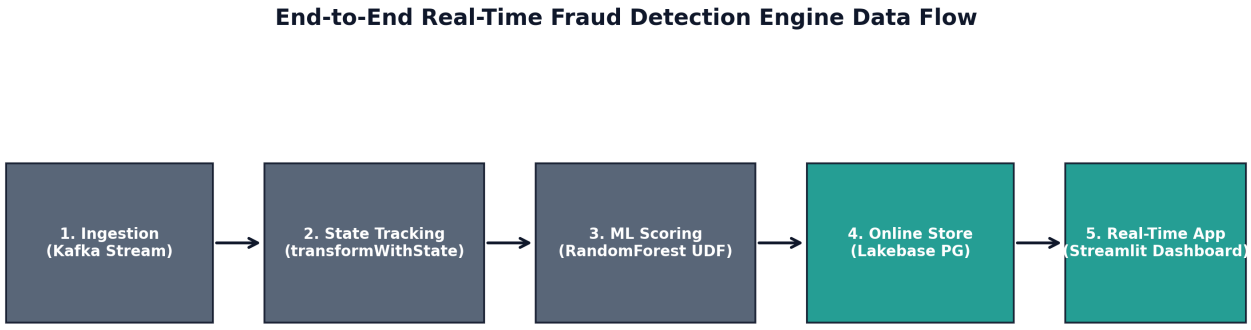


Figure 2. Complete End-to-End Real-Time Fraud Ingestion and Inference Pipeline

Real-Time Pipeline Execution Pattern

1. **Continuous Ingestion:** High-throughput transaction events are streamed directly from Apache Kafka topics into Spark Structured Streaming using the continuous trigger: trigger(realTime=...).
2. **Stateful Feature Extraction:** The transformWithState operator continuously processes incoming events, calculating the per-card transactional velocity (e.g., number of purchases in the last 15 minutes) and geographical velocity (distance between sequential merchant locations).
3. **In-Memory Enrichment:** Transaction metadata is enriched using highly efficient, dictionary-based lookups for cardholder profiles and merchant status, eliminating heavy, latency-inducing BroadcastExchange joins.
4. **ML Model Inference:** Scoring is performed directly inside the stream using a Random Forest classification model trained and versioned in **MLflow**. The model is registered to Unity Catalog and loaded as an optimized Spark User Defined Function (UDF) for inline execution.
5. **Low-Latency Sync:** Scored transactions and dynamic features are simultaneously written to **Databricks Lakebase** (a managed, serverless PostgreSQL database) as an online feature store using a custom foreach sink (LakebaseFeatureWriter), and routed back to downstream Kafka topics for instant payment authorization decisions.

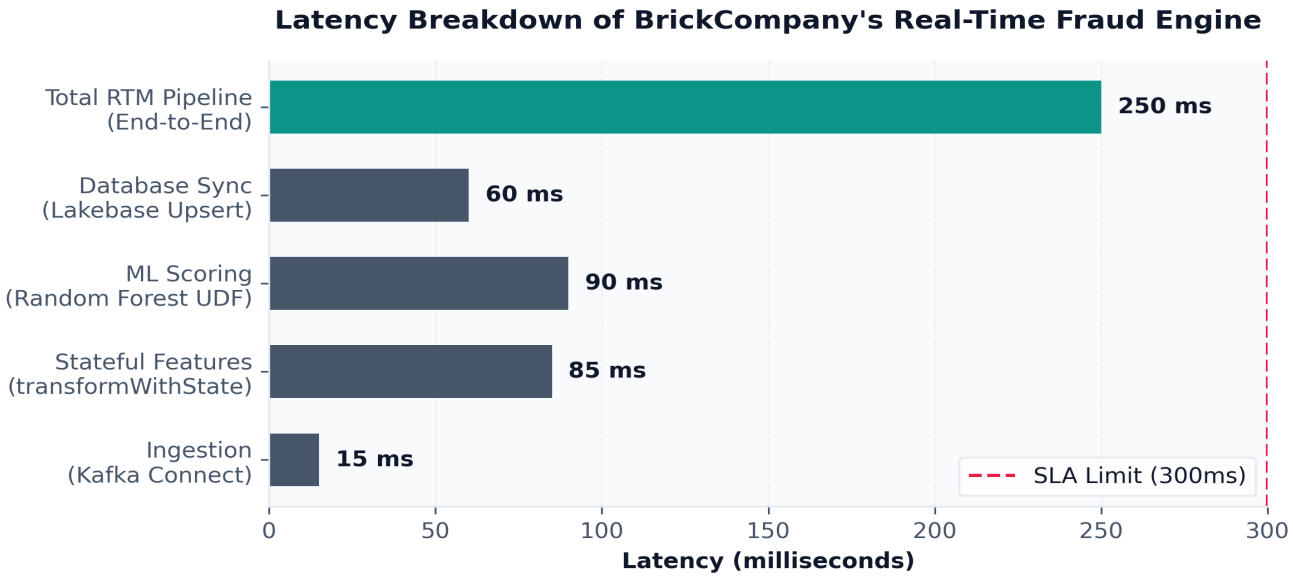


Figure 3. Sub-300ms SLA Latency Budget Breakdown of the Real-Time Fraud Engine

5. Hub-and-Spoke Governance: The 'DataHub' Pattern

Operating a massive, enterprise-scale payment infrastructure requires balancing team autonomy with rigid security guardrails. To prevent operational bottlenecks associated with a monolithic workspace, BrickCompany implements the multi-tenant **DataHub Hub-and-Spoke** architectural design pattern.

In this design model, a **Central Hub Catalog** contains organization-wide, conformed, and read-only data assets (such as master customer profiles, core terminal registers, and centralized audit tables). Independent business units, such as BI Analytics, Fraud Risk, and Data Engineering, operate in isolated **Spoke Workspaces**. Spoke teams possess self-service catalog creation permissions, but easily share curated datasets and discover hub metadata via **Unity Catalog**. Delta Sharing is configured to securely exchange live, up-to-date features across different regional clouds or subscriptions without the need for copy-heavy, brittle ETL pipelines.

> **PCI-DSS & Privacy Compliance Note:** Unity Catalog acts as the single pane of glass for all data assets. It enforces column-level masking (e.g., dynamic redaction of cardholder PAN and CVV numbers) and row-level filters (e.g., restricting analysts to specific regional transaction segments) to comply with strict international payment security standards (PCI-DSS) and local regulatory requirements.

6. Operational Monitoring with Databricks Apps

To close the operational loop, payment risk analysts require instantaneous visibility into engine performance. BrickCompany deploys a real-time, Streamlit-based monitoring application natively hosted as a **Databricks App**.

The application integrates seamlessly with the streaming architecture by utilizing Databricks' native **Lakebase Resource Bindings**. The app reads its database credentials directly from auto-injected environment variables (e.g., PGHOST, PGUSER, PGPORT), completely bypassing manual credential management and eliminating the risk of plaintext password leaks in version control.

The dashboard displays total transactions scored, a visual decision breakdown (APPROVED, FLAGGED, BLOCKED), a dynamic fraud probability distribution curve, and high-resolution tabular views of flagged cards. A 10-second auto-refresh toggle allows analysts to pin a snapshot during active incident investigations or monitor live transactional velocities continuously.

Document Status: RELEASED SPECIFICATION

Date of Issue: September 1, 2026

Prepared by: Gemini Notebook Solutions Team

Version: 1.4 (LTS Production Ready)

Enterprise Architect: Chief Data & AI Officer

Target Environment: Databricks AWS Serverless